

Distributed Cache Invalidation, a review of failure patterns

Problem: keeping a distributed cache consistent with its source of truth is hard because invalidation messages can arrive out of order, get dropped, or race with a write that happens right after the cache was refreshed.

This note reviews three recurring failure patterns seen across several postmortems: the thundering herd on simultaneous expiry, the stale read after a write due to invalidation lag, and the invalidation storm where a burst of writes triggers more invalidation traffic than the cache tier can absorb.

Method: for the thundering herd pattern, adding random jitter to TTLs spreads out expiry so instances don't all miss the cache at the same instant. For stale reads after a write, writing through the cache synchronously (updating the cache in the same request that updates the source of truth) closes the window instead of relying on a separate invalidation message arriving in time.

Result: teams that moved from time based invalidation alone to a mix of jittered TTLs and write through updates saw meaningfully fewer stale read incidents, though write through does add latency to the write path that has to be budgeted for.